

HDOOC 21

Question: Write a menu-driven C program that accepts two positive integers, m and n , where $m \leq n$. Display the following menu and accept the user's choice. Use a switch statement to perform the selected operation:

Print the first 10 multiples of each integer from m to n .

Print the addition table of each integer from m to n , using addends from 1 to 10.

Print the subtraction table of each integer from m to n , using subtrahends from 1 to 10.

Print the results obtained by dividing each integer from m to n by the integers from 1 to 10. Display the results rounded to two decimal places.

Exit the program.

Display an appropriate error message if the user enters an invalid menu choice.

Continue displaying the menu and performing the selected operation until the user chooses the Exit option.

Task1: Algorithm:

1. Start.
2. Declare integer variables m , n , choice, i , and j .
3. Input two positive integers m and n .
4. Check whether $m > 0$, $n > 0$, and $m \leq n$.
5. If the input is invalid, display an error message and stop.
6. Otherwise, repeatedly display the menu:
 - 1 – Multiplication tables
 - 2 – Addition tables
 - 3 – Subtraction tables
 - 4 – Division tables
 - 5 – Exit
7. Read the user's choice.
8. Use a switch statement:
 - **Case 1:** For every integer from m to n , print its first 10 multiples.
 - **Case 2:** For every integer from m to n , add the integers 1 through 10 and display the results.
 - **Case 3:** For every integer from m to n , subtract the integers 1 through 10 and display the results.
 - **Case 4:** For every integer from m to n , divide it by the integers 1 through 10 and display each result to two decimal places.
 - **Case 5:** Display an exit message.
 - **Default:** Display an invalid-choice error message.
9. Repeat the menu until the user selects 5.
10. Stop.

Task2: Test case table:

Test Case	Objective	Input	Choice	Expected Result	Actual Result	Status
TC-01	Test multiplication	m=2, n=3	1	First 10 multiples of 2 and 3	Same as expected	Pass
TC-02	Test addition	m=2, n=3	2	Addition tables of 2 and 3 from +1 to +10	Same as expected	Pass
TC-03	Test subtraction	m=2, n=3	3	Subtraction tables of 2 and 3 from -1 to -10	Same as expected	Pass
TC-04	Test division	m=2, n=3	4	Division by 1-10, rounded to 2 decimal places	Same as expected	Pass
TC-05	Boundary case m=n	m=1, n=1	1	Multiples of 1 only	Same as expected	Pass
TC-06	Invalid menu choice	m=2, n=3	7	Error: Invalid menu choice	Same as expected	Pass
TC-07	Invalid range	m=5, n=2	—	Error for $m > n$	Same as expected	Pass
TC-08	Non-positive input	m=0, n=5	—	Error for non-positive input	Same as expected	Pass
TC-09	Exit option	m=2, n=3	5	Program terminates	Same as expected	Pass

Task3: C Program:

```
#include <stdio.h>
int main()
{
    int m, n, choice, i, j;
    printf("Enter two positive integers m and n (m <= n): ");
    scanf("%d %d", &m, &n);
    if (m <= 0 || n <= 0 || m > n)
    {
        printf("Error: Enter positive integers with m <= n.\n");
        return 0;
    }
    do
    {
        printf("\n===== MENU =====\n");
        printf("1. Print first 10 multiples\n");
        printf("2. Print addition tables\n");
        printf("3. Print subtraction tables\n");
        printf("4. Print division tables\n");
        printf("5. Exit\n");
        printf("===== \n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                for (i = m; i <= n; i++)
                {
                    printf("\nMultiplication table of %d:\n", i);
                    for (j = 1; j <= 10; j++)
                    {
                        printf("%d x %d = %d\n", i, j, i * j);
                    }
                }
                break;
            case 2:
                for (i = m; i <= n; i++)
                {

```

```

        printf("\nAddition table of %d:\n", i);
        for (j = 1; j <= 10; j++)
        {
            printf("%d + %d = %d\n", i, j, i + j);
        }
    }
    break;
case 3:
    for (i = m; i <= n; i++)
    {
        printf("\nSubtraction table of %d:\n", i);
        for (j = 1; j <= 10; j++)
        {
            printf("%d - %d = %d\n", i, j, i - j);
        }
    }
    break;
case 4:
    for (i = m; i <= n; i++)
    {
        printf("\nDivision table of %d:\n", i);
        for (j = 1; j <= 10; j++)
        {
            printf("%d / %d = %.2f\n",
                i, j, (double)i / j);
        }
    }
    break;
case 5:
    printf("\nExiting the program...\n");
    break;
default:
    printf("\nError: Invalid menu choice. "
        "Please enter 1 to 5.\n");
}
} while (choice != 5);
return 0;
}

```

Task4: Compilation and execution:

```
(kali㉿kali)-[~]  
$ vi tables.c  
  
(kali㉿kali)-[~]  
$ cc tables.c -otables  
  
(kali㉿kali)-[~]
```

```
(kali㉿kali)-[~]  
$ ./tables  
Enter two positive integers m and n(m≤n):2 3  
10*10=100  
=====MENU=====  
1.Print first 10 multiples  
2.Print addition tables  
3.Print subtraction tables  
4.Print divison tables  
5.Exit  
=====MENU=====  
Enter your choice:1  
  
Multiplication table of 2:  
2*1=2  
2*2=4  
2*3=6  
2*4=8  
2*5=10  
2*6=12  
2*7=14  
2*8=16  
2*9=18  
2*10=20  
  
Multiplication table of 3:  
3*1=3  
3*2=6  
3*3=9  
3*4=12  
3*5=15  
3*6=18  
3*7=21  
3*8=24  
3*9=27  
=====MENU=====  
1.Print first 10 multiples  
2.Print addition tables  
3.Print subtraction tables  
4.Print divison tables  
5.Exit  
=====MENU=====  
Enter your choice:
```

```
=====MENU=====
1.Print first 10 multiples
2.Print addition tables
3.Print subtraction tables
4.Print divison tables
5.Exit
```

```
=====
Enter your choice:2
```

```
Addition table of 2:
```

```
2+1=3
```

```
2+2=4
```

```
2+3=5
```

```
2+4=6
```

```
2+5=7
```

```
2+6=8
```

```
2+7=9
```

```
2+8=10
```

```
2+9=11
```

```
2+10=12
```

```
14*5=70
```

```
Addition table of 3:
```

```
3+1=4
```

```
3+2=5
```

```
3+3=6
```

```
3+4=7
```

```
3+5=8
```

```
3+6=9
```

```
3+7=10
```

```
3+8=11
```

```
3+9=12
```

```
3+10=13
```

```
=====MENU=====
```

```
1.Print first 10 multiples
```

```
2.Print addition tables
```

```
3.Print subtraction tables
```

```
4.Print divison tables
```

```
5.Exit
```